Nama : Averose Arthur R

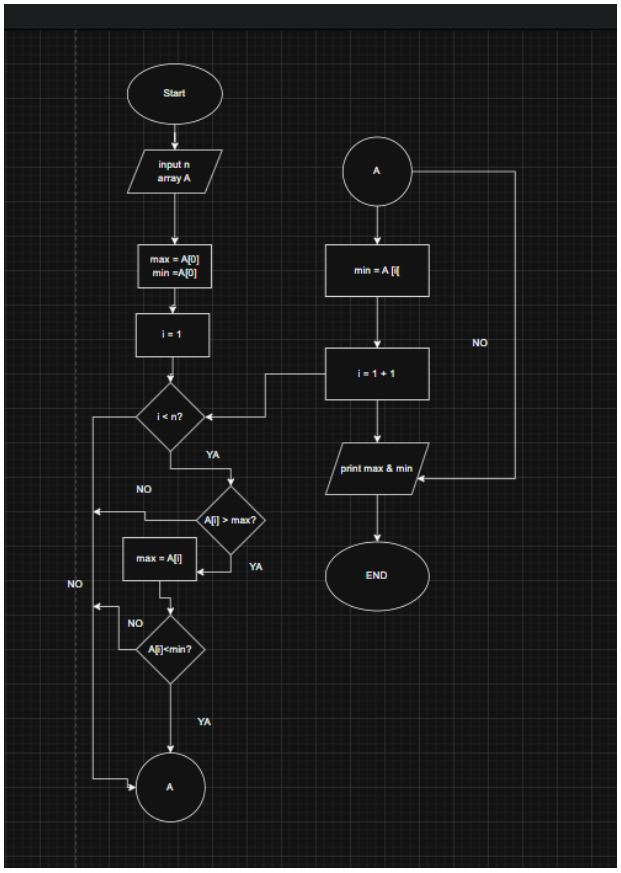
Kelas : TI-1H/04

254107020042

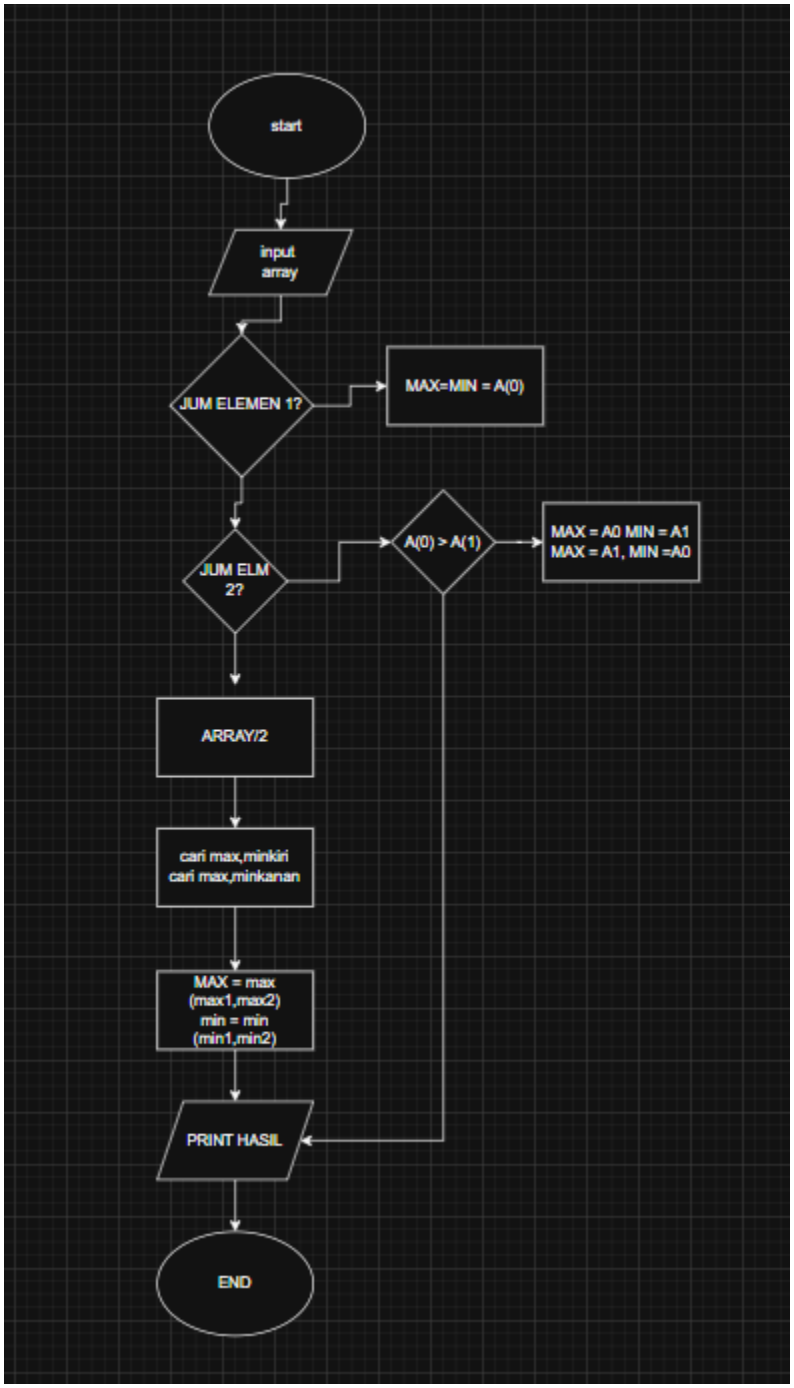
ASD TEORI latihan 5

1. Buatlah flowchart untuk menentukan nilai maksimal dan minimal dari sekumpulan nilai dengan algoritma Brute Force dan Divide Conquer!

Brute Force



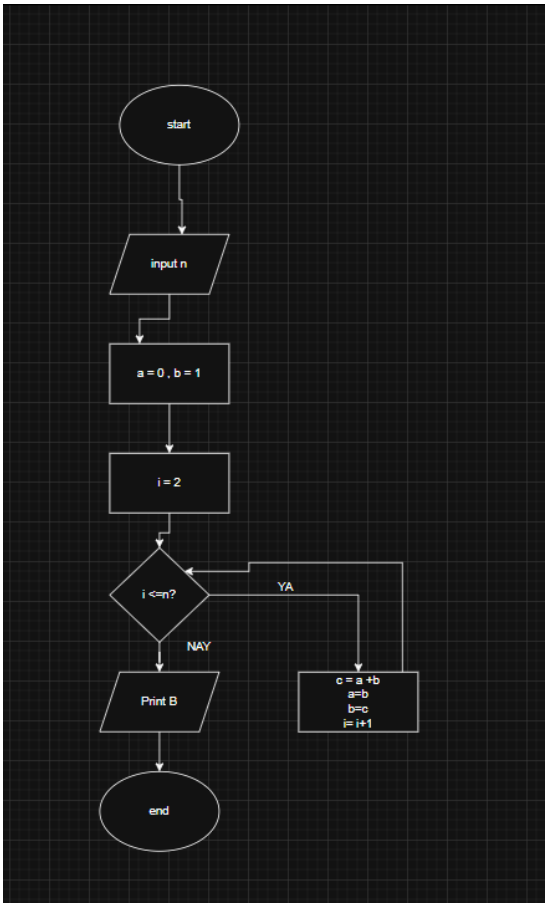
Divide and conq



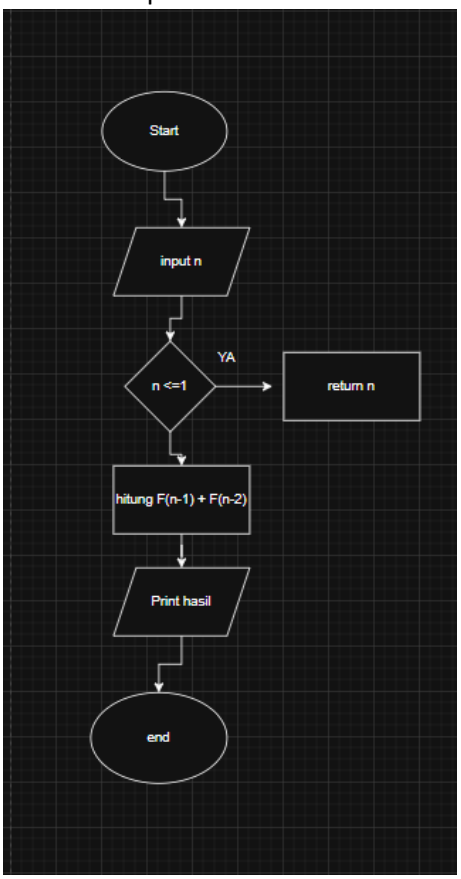
2. Buatlah flowchart untuk menghitung hasil penhitungan deret fibonacci dengan algoritma Brute Force dan Divide Conquer!

   Fibonnaci

   Brute force



Divide nd Cq



3. Tentukan notasi Big O yang sesuai dari kode program berikut!

a.

```
function search(array, value) {
  for (let i = 0; i < array.length; i++) {
    if (array[i] === value) {
      return i;
    }
  }
  return -1;
}
let score = [12, 22, 45, 67, 96];
console.log(search(score, 100));
```

Ans A : gambar **a** menggunakan algoritma **Linear Search**. program melakukan iterasi satu per satu melalui seluruh elemen array menggunakan perulangan for

ketika nilai yang dicari berada di paling akhir atau tidak ada dalam array

program harus mengecek setiap elemen sebanyak n kali, di mana n adalah panjang array.
Notasi Big O: O(n)

b.

```
function binarySearch(array, value) {
  let firstIndex = 0;
  let lastIndex = array.length - 1;
  while (firstIndex <= lastIndex) {
    let middleIndex = Math.floor((firstIndex + lastIndex) / 2);
    if (array[middleIndex] === value) {
      return middleIndex;
    }
    if (array[middleIndex] > value) {
      lastIndex = middleIndex - 1;
    } else {
      firstIndex = middleIndex + 1;
    }
  }
  return -1;
}

let score = [1, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59];
console.log(binarySearch(score, 37));
```

Ans B: gambar **b** menggunakan algoritma **Binary Search**. Algoritma ini bekerja dengan membagi ruang pencarian menjadi dua pada setiap langkah perulangan while

algoritma ini membandingkan nilai tengah (`middleIndex`) dengan target. Jika tidak cocok, ia membuang setengah bagian array yang tidak relevan. Karena jumlah data yang diproses berkurang secara eksponensial (dibagi 2) di setiap langkah, jumlah operasi yang dibutuhkan jauh lebih sedikit dibanding Linear Search.

Notasi big O = O(log n)